

② Ordenação Rápida (Quicksort)

Baseia-se na estratégia de **particionamento**:
dado um elemento p qualquer de um vetor
 $v[e..d]$, queremos encontrar j tal que:

$$\underbrace{v[e..j-1]} \leq p < \underbrace{v[j+1..d]}.$$

não ordenado! $v[j]$ não ordenado!

Ideia do algoritmo de particionamento:

e		k						d	
$\leq p$	$\leq p$	$\leq p$	$> p$	$> p$	?	?	?	...	p
			j						

Ex.:

0	1	2	3	4	5	6	7	$p=3$
1	-2	2	0	3	4	6	7	
				j				p_k

Ex.:

-3	-1	-2	0	3	1	2	7	$p=0$
			j					k

$p = v[d]$, $k = 0$, $j = 0$.

$v[k] \leq p$? Se sim, troca $v[k]$ com $v[j]$ e incrementa j .

Se não, incrementa k . $\rightarrow$ Repete até que $k = d$.

Quando $k = d$, troca $v[j]$ com $v[d]$

```

int PARTICIONA(int *v, int e, int d){
    int j, k;
    FOR(j=0, k=0; k<d; k++){
        IF(v[k] <= v[d]){
            TROCA(&v[k], &v[j]); j++;
        }
    }
    TROCA(&v[j], &v[d])
    RETURN;
}

```

```

void QUICK SORT(int *v, int e, int d){
    IF(e<d){
        int j = PARTICIONA(v, e, d);
        QUICK SORT(v, e, j-1);
        QUICK SORT(v, j+1, d);
    }
}

```

```

na main QUICK SORT(v, 0, n-1);

```

```

void quickSort(int *v, int e, int d){
    if(e < d){
        (*
        int j = PARTICIONA(v, e, d);
        quickSort(v, e, j-1);
        quickSort(v, j+1, d);
    }
}

```

```

int PARTICIONA(int *v, int e, int d){
    int j = e, p = v[d];
    for(int k = e; k <= d; k++){
        if(v[k] <= p) TROCA(&v[k], &v[j++]);
    }
    return j-1;
}

```

$\leq p$	$\leq p$	$\leq p$	$> p$	$> p$	$> p$	...	...	...	p
		j	j	k					

Retorna j tal que

$$v[e..j-1] \leq p = v[j] < v[j+1..d]$$

Ex.:

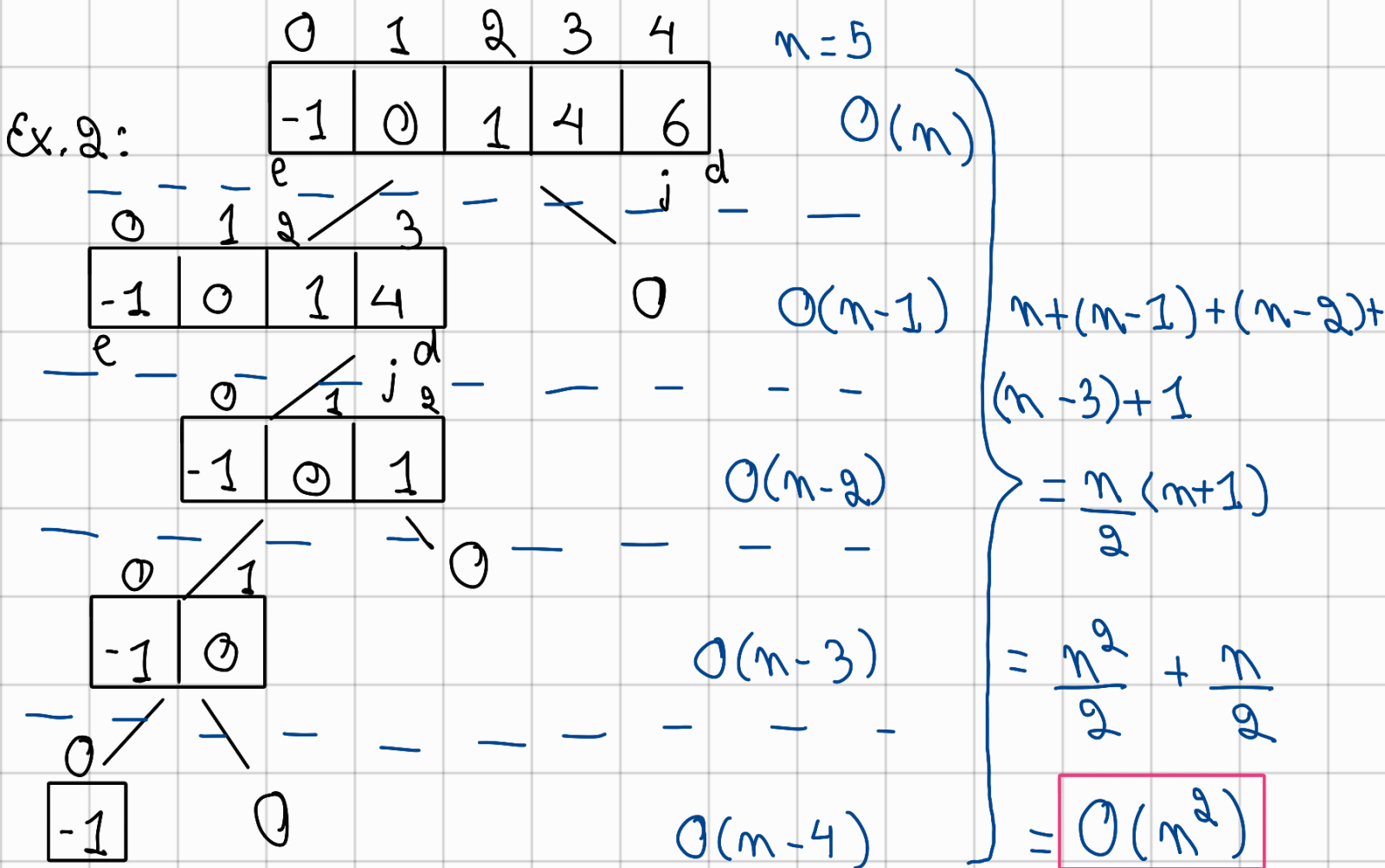
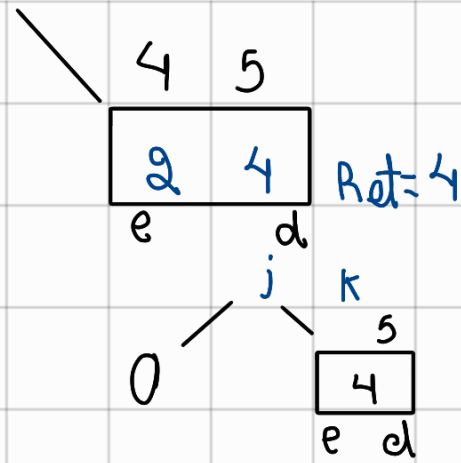
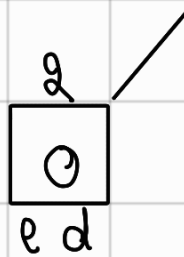
0	1	2	3	4	5
-3	-1	0	1	2	4

 e
 j
 d
 k
ret part = 1

0
-3
 e d

2 3 4 5
0 1 4 2
 e j d k Retorno = 3

Árvore de Recursão



Para reduzir a chance de ocorrência da pior caso, devemos adotar uma estratégia de pivoteamento que preserve que o pivô seja o maior elemento do vetor.

Uma estratégia muito usada é a **mediiana de 3**:

- ① Calcula o **valor médio** entre $V[e]$, $V[(e+d)/2]$ e $V[d]$
 - ② Posiciona o **valor médio** em $V[d]$ (o antigo $V[d]$ na posição do **valor médio**).
- Inserimos uma função que faça isso no quicksort em (*)

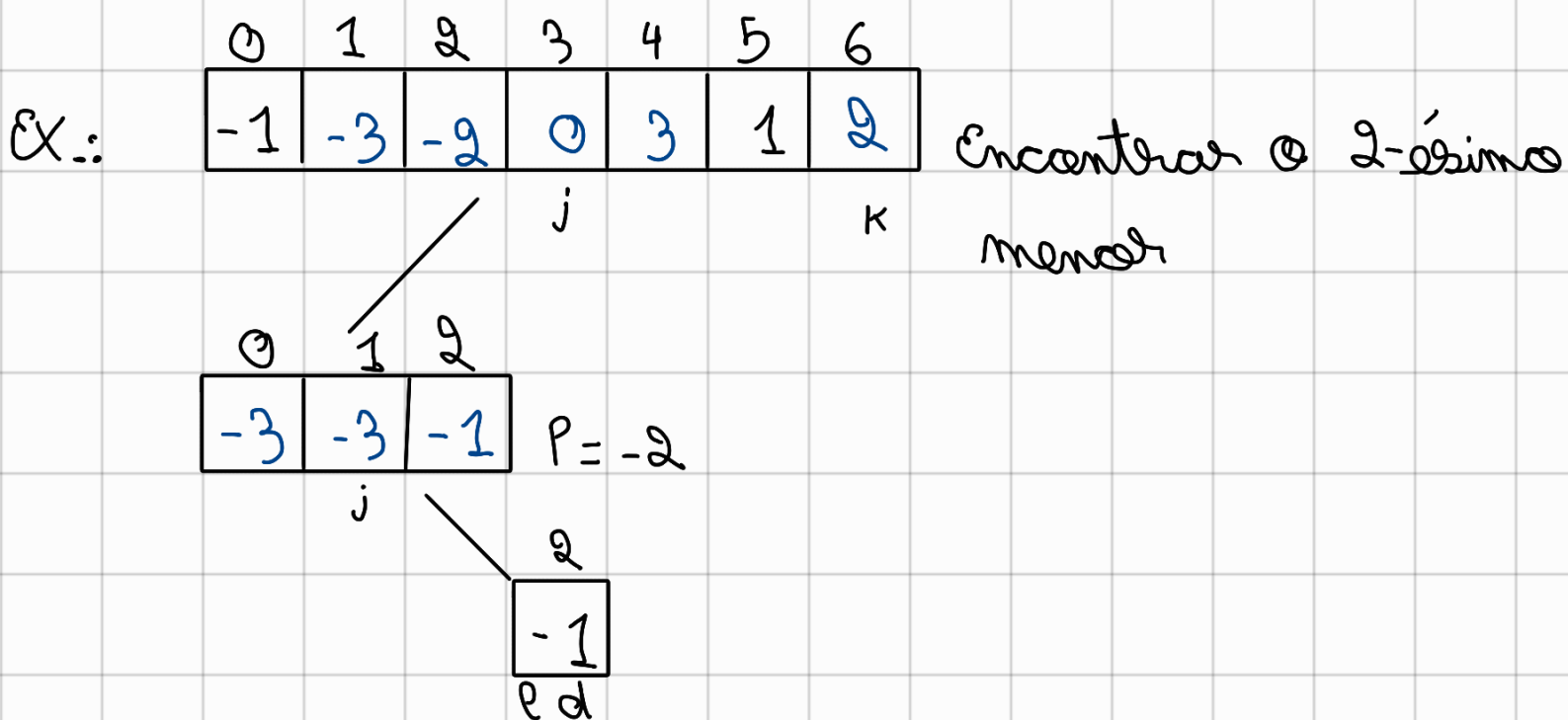
Há inúmeras outras estratégias como essa, para todas, é possível demonstrar que o caso médio é $O(n \lg n)$ (mas continua $O(n^2)$ no pior caso).

Quicksort, apesar do custo de pior caso, comporta-se muito bem na prática.

Obs.: mergesort é um algoritmo **estável**, mas quicksort **não** é.

Problema: Encontre o k -ésimo menor elemento de um vetor. ($0 \leq k < n$).

$k=0$ seria encontrar o menor elemento.



```

VOID QUICKSELECT (INT*V, INT e, INT d, INT K){
    IF (e < d){
        MEDIANA3 (V, e, d);
        INT j = PARTICIONA (V, e, d);
        IF (K < j) QUICKSELECT (V, e, j-1, K);
        IF (K > j) QUICKSELECT (V, j+1, d, K);
    }
}

```

Custo: $n + \frac{n}{2} + \frac{n}{4} + \dots + 1$

$$\leq n + \frac{n}{2} + \frac{n}{4} + \dots = \frac{0,1}{1-0,5} = \frac{\frac{n}{1}}{2} = 2n \therefore \underbrace{O(n)}_{\text{caso médio}}$$

QUICKSELECT não apenas seleciona o k -ésimo menor mas reorganiza o vetor inteiro tal que:

$$V[0 \dots k-1] \leq V[k] \leq V[k+1 \dots d].$$

Então pode ser interessante para resolver problemas do tipo:

Ex.: Suponha que você tem uma lista muito grande de produtos e deseja exibi-los num site, **10** por página, ordenados por preço.

Solução: para a pág. 1:

$$P.1 : V[0 \dots 9].$$

$$P.2 : V[10 \dots 19].$$

⋮

$$P.i : V[10(i-1) \dots 10i-1]$$

QUICKSELECT($V, 0, n-1, 10^*(i-1)$);

QUICKSELECT($V, 10^*(i-1)+1, n-1, 10^*(i-1)$);

QUICKSORT($V, 10^*(i-1), 10^*i-1$);